

FRS

FRS DOCUMENT:

Project K

Revision History

Rev. No.	Date	Revision Descriptions	Signature (Rytsense)	Signature (Knotopian)
1	07-04-2026	Final copy Issued for Submission		

1) Introduction

The purpose of this document is to define the **functional and technical requirements** of the AI-powered eLearning and Interview Platform, consisting of:

- A **B2B institutional platform** for colleges and universities
- A **B2C AI Interview platform** for individual users

The system enables:

- AI-driven course generation from academic content
- AI-powered tutoring and assessments
- Voice-based AI interview simulation with evaluation and reporting

This document serves as a **reference for product, engineering, QA, and stakeholders** to ensure aligned development and delivery.

2. Purpose

The objective of this platform is to:

- Transform static academic content into **interactive AI-driven learning experiences**
- Provide **personalized, on-demand tutoring**
- Enable **automated interview preparation with real-time evaluation**
- Offer **data-driven insights** for students, faculty, and institutions

Key Outcomes:

- Improve student learning efficiency
- Increase placement readiness
- Provide institutions with actionable analytics
- Create a scalable AI-first education ecosystem

3. Overall Description

3.1 Product Perspective

The system is a **multi-tenant SaaS platform** consisting of:

B2B Platform:

- Super Admin Portal (Platform Owner)
- College Admin / Faculty Portal
- Student Portal

B2C Platform:

- Student Portal (Standalone Interview System)
- Super Admin Portal (Global control)

Core Systems:

- AI Content Generation Engine (RAG-based)
- AI Interview Engine (voice-based)
- AI Tutor Engine (contextual QnA)
- Scoring & Report Engine
- Notification System

3.2 Operating Environment

Frontend:

- Web-based application (Desktop-first)
- Built using ReactJS
- Supported browsers: Chrome, Edge, Safari (latest versions)

Backend:

- Python FastAPI (REST APIs)
- Asynchronous processing supported

Infrastructure:

- Cloud: AWS / GCP
- Storage: AWS S3 (media, content, recordings)
- Database: PostgreSQL
- Vector DB: Pinecone / Weaviate

AI Stack:

- LLM APIs: OpenAI / Anthropic / Google
- STT: Whisper / Deepgram
- TTS: ElevenLabs / Deepgram
- Real-time: LiveKit / Vapi

3.3 Design & Implementation Constraints

- System must be **multi-tenant with strict data isolation**
- All AI responses must comply with **guardrails (safe, contextual outputs)**
- Voice-based systems must maintain **low latency (<3 seconds response cycle)**
- Avatar systems must not degrade performance
- Platform must support **horizontal scaling**

- Data retention:
 - Audio/transcripts: 30 days
 - Resume: until deletion

FR. Id		Requirement Statement
AI Interview Engine		
FR-1: Interview Initializat ion		User selects: • Role (e.g., Frontend Developer) • Experience level (Fresher / Junior) • Interview type (Mock / Full-length) System loads: • Question bank (AI-generated) • Resume (if available)
FR-2: Voi ce Inte ract ion		• System asks questions via TTS • User responds via voice (STT) • No manual text input (B2C constraint) We can have a fallback scenario for text input
FR-3: Adaptive Questioni ng		• Questions generated dynamically based on: ◦ Previous answers ◦ Resume ◦ Skill tags • Follow-up logic: ◦ Max 1 (mock), Max 2 (full interview)
FR-4: Context Retention		• System maintains: ◦ Conversation history ◦ Answer quality signals • Avoids repetition
FR-5:		Mock: <5 mins

Time-Based Flow		Full: up to 15 mins Countdown visible
FR-6: Session Completion •		- Ends when: - Time limit reached - Questions completed - User drops
System Flow (CRITICAL)		- User clicks "Start Interview" - System initializes session - AI introduces interview - Question loop begins: - Ask question - Capture answer - Evaluate - Decide next step - Closing statement - Report generated Step 1: Session Init - Create session_id - Load: - user_profile - resume_embedding (if exists) Step 2: Question Generation Prompt input: - Role - Skills - Previous answers - Difficulty level

		LLM Output: <pre>{ question: "", type: "technical/behavioral", difficulty: "easy/medium/hard", follow_up_possible: true }</pre> Step 3: Voice Pipeline TTS → Play Question ↓ User Speaks ↓ STT → Transcript ↓ LLM Evaluation ↓ Next Question Decision Step 4: Evaluation (per answer) Output format: <pre>{ communication_score: 0-10, technical_score: 0-10, behavioral_score: 0-10, relevance_score: 0-10, confidence_score: 0-10, feedback: "", improvement: "" }</pre> Step 5: Decision Engine • If answer strong → next question • If weak → follow-up • If time low → skip follow-up
Module 2 : Content Engine		

FR-1: Content Upload & Tagging		• Faculty/Admin uploads: ◦ PDF / PPT / Text / Question bank • Mandatory tagging: ◦ Department ◦ Program ◦ Semester ◦ Subject ◦ Chapter
FR-2: Content Processing Pipeline (Async)		Once uploaded: 1. File parsing (extract text) 2. Chunking (semantic segmentation) 3. Embedding generation 4. Storage in vector DB (chapter-scoped)
FR-3: Knowledge Base Creation (RAG)		• Each chapter = isolated knowledge base • Retrieval must be: ◦ Contextual ◦ Relevant ◦ Non-cross-chapter
FR-4: AI Video Generation		• AI generates: ◦ Script (structured explanation) ◦ Scene breakdown • Converted into: ◦ Avatar-based video
FR-5: SME Review Workflow		• Generated videos: ◦ Must be reviewed before publishing • Actions: ◦ Approve

		○ Reject ○ Request regeneration
FR-6: MC Q Gen era tio n		• AI generates: ○ Multiple-choice questions ○ Answer + explanation • SME approval required before activation
3. U se r Fl o w (F ac ul ty)		1. Upload content 2. Assign to chapter 3. Trigger AI generation 4. Wait for processing 5. Review: ○ Video ○ MCQs 6. Approve → visible to students Step 1: Content Ingestion • Extract text using parser • Clean: ○ Remove noise ○ Normalize formatting Step 2: Chunking Strategy • Chunk size: ~300–500 words • Maintain: ○ Semantic boundaries ○ Topic coherence Step 3: Embedding • Generate embeddings per chunk • Store with metadata: ○ chapter_id ○ subject_id Step 4: Video Script Generation

		LLM Input: • Chapter content (retrieved chunks) • Academic level • Learning objective LLM Output: • Structured script: ◦ Introduction ◦ Concept explanation ◦ Examples ◦ Summary Step 5: Video Rendering • Script → TTS → Avatar rendering • Output: ◦ Video file (MP4) ◦ Duration: 5–10 mins per chapter Step 6: MCQ Generation LLM generates: • Question • 4 options • Correct answer • Explanation
QnA AI Tutor MODULE		
FR-1: Session Start		Student opens QnA after video Context: • Chapter ID • Subject • User session
FR-2: Voice Interaction		• Student asks question via voice • System responds via TTS
FR-3:		• Retrieve relevant chunks • Generate answer using:

Contextual Answering (RAG)		○ Retrieved content ○ Conversation history
FR-4: Multi-turn Conversation •		● Maintain session memory ● Allow: ○ Follow-up questions ○ Clarifications
FR-5: Interruption Handling •		● Student can interrupt AI mid-response ● System must: ○ Stop TTS ○ Process new query
User Flow		1. Student opens QnA 2. Asks a question (voice) 3. AI responds 4. Student asks follow-up 5. Session continues until exit 4. System Flow Step 1: Voice Input ● Capture audio

		• Convert using STT Step 2: Query Processing • Clean query • Detect language • Map to chapter context Step 3: Retrieval (RAG) • Fetch top-k relevant chunks • Ensure: ◦ Same chapter ◦ High similarity Step 4: Answer Generation LLM Input: • User question • Retrieved chunks • Conversation history Output: • Structured explanation • Examples (if needed) Step 5: Voice Output • Convert response → TTS • Play audio
Scoring Engine		
<u>FR-2: Per-Answer Score</u>		• <u>Every answer gets:</u> ◦ <u>Score (0–10) per category</u> ◦ <u>Feedback</u> ◦ <u>Improvement suggestion</u>

<u>ring</u>		
FR-3: Session-Level Aggregation		• Final score = weighted aggregation of all answers • Category-wise averages maintained
FR-4: Signal-Based Evaluation		Scoring must combine: AI Signals: • Semantic correctness • Depth of answer • Logical reasoning Rule-Based Signals: • Answer length • Pause duration • Speech speed • Filler word frequency
User Flow		1. User answers question 2. System evaluates instantly 3. Scores stored internally 4. Used later for report generation 4. System Flow Step 1: Input Collection Inputs: • Transcript (from STT) • Audio metadata: ○ duration ○ pauses

		○ speech rate ● Question context Step 2: Pre-Processing ● Clean transcript ● Normalize text ● Remove noise Step 3: AI Evaluation LLM evaluates: ● Accuracy ● Completeness ● Structure ● Examples used Output: ● Category scores ● Qualitative feedback Step 4: Rule-Based Scoring System calculates: ● Words per minute (speech speed) ● Silence duration ● Filler words: ○ “um”, “uh”, “like” ● Over-length / under-length answers Step 5: Score Normalization ● Combine AI + rule signals ● Normalize to 0–10 scale
REPORT ENGINE		
FR-1: Report Generation Trigger		Generated immediately after interview completion Available via: ● Dashboard ● Email / WhatsApp

FR-2: Report Components**1. Overall Score**

- Final weighted score
- Performance label:
 - Excellent / Good / Average / Needs Improvement

2. Category Breakdown

- Scores per category
- Visual indicators (progress bars)

3. Strengths

- Top 3 strong areas
- Based on scoring trends

4. Weaknesses

- Top 3 weak areas
- Must be specific (not generic)

5. Question-wise Feedback

For each question:

- User answer summary
- Score
- Feedback
- Suggested improvement

6. Improved Answers

- AI generates:
 - Ideal answer for weak questions
- Must:
 - Be structured
 - Be realistic (not perfect textbook)

7. Transcript

- Full conversation log
- Time-stamped

8. Performance Trends

		• Last N sessions comparison: ◦ Score progression ◦ Category improvement
Use r Fl o w		1. Interview ends 2. Report generated 3. User views: ◦ Summary → details → improvements 4. User downloads PDF 4. System Flow Step 1: Data Aggregation Collect: • All scores • All answers • All feedback Step 2: Insight Generation LLM analyzes: • Patterns across answers • Strength clusters • Weakness clusters Step 3: Improved Answer Generation For weak answers: • Generate ideal response using: ◦ Question ◦ Expected knowledge level Step 4: Report Structuring Organize into: • Summary • Detailed breakdown • Actionable insights

		Step 5: PDF Generation Convert report → downloadable PDF
	Module	AUTHENTICATION & USER MANAGEMENT
FR-1: Multi-Mode Authentication		B2B: - Email + Password login - Optional OTP verification (first login) B2C: - WhatsApp OTP login (primary) - Mobile number-based authentication
FR-3: Session Management		- JWT-based authentication - Token expiry (e.g., 24 hrs) - Refresh token mechanism
FR-4: Profile Management		- Users can: - Update profile - Add skills - Upload resume (B2C + B2B students)
FR-5: Bulk User Management		- CSV upload (max 100 users per batch) - Auto account creation - Credential delivery via email/SMS

ement ent (B2 B)		
FR-6: Acc oun t Acti vati on		New users receive: ● Email/SMS with credentials First login → mandatory password reset
	Module	SUBSCRIPTION & BILLING (B2C)
FR-1: Sub scri ptio n Pla ns		Standard: ● Limited interviews ● Basic reports Premium: ● Extended interviews ● Advanced reports ● Full avatar usage
FR-2: Usa ge Tra		Track per user: ● Mock interviews used ● Full interviews used

cki ng		● Remaining credits
FR-3: Add -On Cre dits		Users can purchase: ● Extra interviews Credits added instantly
FR-4: Sub scri ptio n Life cycl e		States: ● Active ● Expired ● Cancelled
FR-5: Pay me nt Inte gra tio n		● Payment gateway (Razorpay / Stripe) ● Handle: ○ Success ○ Failure ○ Retry
	MODULE 8	NOTIFICATION SYSTEM
FR-1: Not ific atio n		● Email (primary) ● WhatsApp (secondary / OTP / alerts)

Channels		
	Module 9	AI AVATAR SYSTEM
FR-1: Avatar Rendering Modes		Mode 1: Real-Time Streaming Avatar ● Low-latency streaming ● Used in: ○ Interview sessions ○ QnA sessions Mode 2: Pre-Rendered Avatar ● Used when: ○ Real-time fails ○ Network is weak
FR-2: Voice-Synced Avatar Speech		Avatar must: ● Speak using TTS-generated audio ● Synchronize lip movement with speech Supported engines: ● HeyGen / similar
FR-3: Conversational Tur		● Avatar must: ○ Wait for user input ○ Speak only when appropriate ● Must support: ○ Turn-based interaction

n Ha ndli ng		○ Dynamic response timing
FR-4: Rea l-Ti me Inte rru ptio n Ha ndli ng		● User can interrupt avatar while speaking ● System must: ○ Stop avatar speech immediately ○ Pause video stream ○ Capture new input
FR-6: Ses sio n Syn chr oni zati on		Avatar must stay in sync with: ● AI response generation ● Voice playback ● User interaction state
User Flo w		Interview Mode 1. Avatar appears on screen 2. Introduces interview 3. Asks question (video + voice) 4. User answers 5. Avatar reacts → next question QnA Mode 1. Avatar greets user

		2. User asks question 3. Avatar explains concept 4. User interrupts → follow-up
System Flow		Step 1: AI Response Generation • LLM generates response text Step 2: TTS Conversion • Text → audio stream Step 3: Avatar Rendering Two possible pipelines: ◆ Option A: Streaming Avatar API (Preferred) LLM Response ↓ TTS Audio Stream ↓ Avatar API (lip-sync rendering) ↓ WebRTC Stream → Frontend ◆ Option B: Chunked Rendering Text → Split into chunks ↓ Each chunk → TTS + Avatar render

		↓ Stream sequentially Step 4: Frontend Playback ● Render avatar video ● Sync audio + video ● Maintain low latency
--	--	--

AI Educational Video Generation System

AI Educational Video Generation System

– Cost Estimation (AWS)

1. System Pipeline

- Admin uploads PDF
- PDF Text Extraction (PyMuPDF / pdfplumber)
- Content Structuring (LLM – Mistral / Phi3)
- Video Script Generation (LLM)
- Visual Scene Generator (Manim / Mermaid / Graphviz)
- Image Generator – Optional (Stable Diffusion)
- Voice Narration (Piper TTS / Coqui TTS/ Edge TTS)
- Video Rendering (Manim / MoviePy / FFmpeg)
- Final Educational Video Output

2. Software Cost

Tool	License Cost
PyMuPDF / pdfplumber	Free (Open Source)
Mistral / Phi3 LLM	Free (Open Source)
Manim	Free (Open Source)
Mermaid	Free (Open Source)
Stable Diffusion	Free (Open Source License)
Piper TTS / Coqui TTS	Free (Open Source)
FFmpeg / MoviePy	Free (Open Source)

3. AWS Infrastructure Components

3.1 API Server

Instance Type: t3.medium

Estimated Monthly Cost: ~\$30 (~₹2500)

3.2 Worker Server (CPU Processing)

Instance Type: c6i.large

Estimated Monthly Cost: ~\$60 (~₹5000)

3.3 GPU Server (Stable Diffusion)

Instance Type: g4dn.xlarge

GPU: NVIDIA T4

Hourly Cost: ~\$0.526

Monthly Cost (24/7): ~\$384 (~₹32,000)

Recommended Usage: On-demand only when generating images.

4. GPU Cost Optimization

GPU Usage per Day	Estimated Monthly Cost
2 hours/day	~\$32 (~₹2600)
4 hours/day	~\$64 (~₹5300)
6 hours/day	~\$96 (~₹7900)

5. Additional Services

Service	Estimated Monthly Cost
AWS S3 Storage (100GB)	~\$2–\$5
PostgreSQL RDS (db.t3.micro)	~\$15

Redis (ElastiCache)	~\$15
---------------------	-------

6. Total Estimated Monthly Cost (Startup Stage)

API Server: \$30

Worker Server: \$60

GPU (4 hrs/day): \$64

Database: \$15

Redis: \$15

S3 Storage: \$5

Total Estimated Monthly Cost: ~\$189 (~₹15,500)

7. Minimum Startup Architecture (Low Cost Option)

1 CPU Server (EC2) (c6i.large) 32.324400 USD - INR 3015.42

+ On-demand GPU 246.740000 USD/ Per Month - INR 23017.42 /Per Month

GPU Cost Per Hour

- **USD:**

$246.74 \div 730 \approx 0.34$

≈ \$0.34 per hour

- **INR:**

$23,017.42 \div 730 \approx 31.53$

≈ ₹31.5 per hour

1 image generation (5 minutes GPU use)

→ $₹31 \div 12 \approx ₹2.5$

10 images

→ ₹25

100 images

→ ₹250

+ S3 Storage(50GB \$3/Per Month). - INR 279.86.

AI-Based Educational Video Generation System

Project Approach Document

1. Objective

Build a system that automatically converts educational PDFs (e.g., engineering textbooks) into short animated educational videos.

The system should:

- Accept PDF uploads from an admin.
- Extract and structure educational content.
- Divide long chapters into smaller segments (e.g., 5-minute sections).
- Generate visual explanations (diagrams, animations, slides).
- Generate voice narration.
- Render the final educational video automatically.

2. High-Level Workflow

Admin uploads PDF



PDF Text Extraction



Content Structuring using LLM



Video Script Generation



Visual Scene Generation



Image / Diagram Generation



Voice Narration



Video Rendering



Final Educational Video

3. System Architecture Components

3.1 API Server

Handles user requests, authentication, PDF uploads, and job management.

3.2 Background Worker

Processes heavy tasks such as content generation, diagram rendering, and video rendering.

3.3 Queue System

Manages background jobs to ensure tasks run sequentially without overloading the system.

3.4 Storage

Stores uploaded PDFs, generated images, and final videos in S3.

3.5 GPU Worker (Optional)

Runs image generation models such as Stable Diffusion when required.

4. Detailed Processing Pipeline

Step 1 – PDF Upload

Admin uploads educational PDF content.

Step 2 – Text Extraction

Extract text using PyMuPDF or pdfplumber.

Step 3 – Content Structuring

Use LLM models (Mistral / Phi-3) to:

- Identify chapters
- Summarize sections
- Break content into 5-minute video segments

Step 4 – Script Generation

Generate narration script and scene descriptions.

Step 5 – Visual Scene Generation

Select visualization method:

- Manim for animations
- Mermaid for diagrams
- Graphviz for flowcharts

Step 6 – Image Generation (Optional)

Generate illustrative images using Stable Diffusion.

Step 7 – Voice Narration

Convert script to audio using Piper TTS or Coqui TTS.

Step 8 – Video Rendering

Combine visuals and narration using:

- Manim
- MoviePy
- FFmpeg

5. Technology Stack

Backend: Python (FastAPI)

Queue System: Redis + Celery

LLM Models: Mistral / Phi-3

PDF Processing: PyMuPDF / pdfplumber

Animation Engine: Manim

Diagram Engine: Mermaid / Graphviz

Image Generation: Stable Diffusion XL

Text-to-Speech: Piper TTS / Coqui TTS

Video Rendering: MoviePy / FFmpeg

Storage: AWS S3

Database: PostgreSQL (AWS RDS)

6. Deployment Architecture (AWS) – MVP Setup

- **Compute Server:**

- Single EC2 instance
- Instance Type: c6i.large
- Runs **API server + Worker jobs + Redis queue** • GPU

- **Processing (On-demand):**

- Instance Type: g4dn.xlarge
- Used only when **Stable Diffusion image generation** is required
- Start → generate image → stop instance •

- **Storage:**

- **Amazon S3**
- Stores PDFs, generated images, audio files, and final videos •

- **Database:**

- **Amazon RDS (PostgreSQL) • Queue**

- **System:**

- **Redis**

- Handles background processing jobs

7. Expected Benefits

- Automatic conversion of textbooks into video content
- Scalable architecture
- Low operational cost using open-source tools
- Flexible visualization pipeline for multiple subjects

AI Interview Module

1. INTERVIEW MODULES

1. Audio-Based Interview

- Duration: 5 minutes
- Output: Voice-only interaction
- Avatar Rendering: Not included

Optimized for:

- Low cost
- High concurrency
- Fast response

2. Avatar-Based Interview

- Duration: 15–20 minutes
- Output: Real-time talking avatar
- Technology: MuseTalk + GPU

Optimized for:

- Realistic human interaction
- Premium experience

2. HIGH-LEVEL ARCHITECTURE (COMMON CORE)

Flow:

- User Speech
- Speech-to-Text (Whisper)
- LLM Interview Engine (OpenAI Small Model)
- Text-to-Speech (OpenAI TTS)
- Audio Output

3. MODE-SPECIFIC FLOW

AUDIO-BASED INTERVIEW FLOW (5 min)

- User → Mic Input

- Streaming STT (Whisper)
- LLM (Question + Follow-up)
- TTS (Generate Voice)
- Return Audio to User

AVATAR-BASED INTERVIEW FLOW (15–20 min)

- User → Mic Input
- Streaming STT (Whisper)
- LLM Engine
- TTS (Audio Generation)
- MuseTalk GPU Service
- Lip-Synced Frames
- WebRTC Streaming to Frontend

MuseTalk Role

- Converts TTS audio → Talking avatar

Uses:

- VAE (Image Encoding)
- Whisper-based Audio Embeddings
- UNet with Cross-Attention

4. UNIFIED INTERVIEW ENGINE

Both modes use the same backend logic.

Interview Engine Responsibilities:

- Dynamic question generation
- Adaptive follow-up logic
- Context tracking
- Difficulty adjustment
- Avoid repetition

Flow Logic

Audio Mode:

- 3–5 questions
- Minimal follow-ups

Avatar Mode:

- 8–12 questions
- Dynamic follow-ups
- Full interaction experience

5. STREAMING DESIGN

Audio Mode

- Protocol: WebSocket
- Lightweight streaming

Avatar Mode

- Protocol: WebRTC

Required for:

- Video frames
- Low latency streaming

6. MICROSERVICE ARCHITECTURE

- API Gateway
- Auth Service
- Session Service
- Interview Engine (LLM)
- STT Service (Whisper)
- TTS Service (OpenAI TTS)
- Avatar Service (MuseTalk GPU)
- Streaming Service (WebRTC)
- Report Service

7. INPUT CONFIGURATION (FRONTEND INTEGRATION)

Users can initiate interviews using two methods:

Option 1: Resume Upload

- User uploads resume from frontend
- Resume is processed using parsing/OCR
- Extracted data:
 - Skills
 - Experience
 - Projects
- Interview Engine generates personalized questions based on resume

Option 2: Interview Position Selection

- User selects job role (e.g., Backend Engineer, Data Scientist)
- System loads predefined skill requirements
- Interview Engine generates role-based questions

Combined Logic

- If both resume + role are provided:
 - Resume → personalization
 - Role → structure & difficulty
- Ensures highly relevant and adaptive interviews

8. OUTPUT & REPORTING (Using Reasoning LLM from Open AI)

- Transcripts
- AI-generated evaluation:
 - Communication skills
 - Technical accuracy
 - Confidence
- Final report generation

9. FallBack Mode For Avatar

1. Play a pre rendered video of the same avatar.

Cost:

Audio:

Component	Description	Cost (₹)	Justification
LLM (Interview Engine)	Question generation and follow-ups	₹1 – ₹2	Small model with low token usage
STT (Whisper)	5 min speech-to-text	₹3	Whisper pricing is about \$0.006 per minute
TTS	5 min speech output	₹2 – ₹3	Approx \$0.015 per minute audio generation
AWS Infra	Backend, database, streaming, API	₹4 – ₹5	Includes EC2, RDS, WebSocket, load balancer (shared infra cost)
Buffer / Retry	Failures and reprocessing	₹1 – ₹2	Production safety margin
Report (LLM + PDF)	Evaluation and report generation	₹8 – ₹12	Reasoning model + transcript processing
TOTAL COST	End-to-end per session cost	₹18 – ₹27	Includes interview and report

Component	Description	Cost (₹)	Justification
LLM (Interview Engine)	Long conversation and adaptive logic	₹3 – ₹5	Higher token usage compared to audio
STT (Whisper)	15–20 min speech-to-text	₹6 – ₹8	Based on per-minute transcription cost
TTS	15–20 min speech output	₹5 – ₹6	Continuous audio generation
GPU (MuseTalk)	Avatar rendering	₹12 – ₹15	GPU hourly cost distributed per

			session
AWS Infra	Backend, database, streaming, API	₹5 – ₹6	Includes EC2, RDS, WebRTC, load balancer (shared infra cost)
Buffer / Retry	Sync issues and reprocessing	₹2 – ₹3	Real-time system reliability
Report (LLM + PDF)	Evaluation and report generation	₹8 – ₹12	Reasoning model + transcript processing
TOTAL COST	End-to-end per session cost	₹33 – ₹42	Includes interview and report